



Parallélisme à base de thread

(PABT 2025-2026)

TD2

Programmation par tâches
OpenMP

teodora.hovi@cea.fr

Un rapport au format PDF comprenant les réponses aux questions textuelles ainsi que des explications détaillées sur le code est attendu (voir lien en fin d'énoncé).

L'utilisation de chaque directive OpenMP et chaque clause associée devra ainsi trouver justification dans le rapport.

Rappels

- Pour compiler un programme C utilisant OpenMP avec le compilateur gcc :
`% gcc -fopenmp source.c -o exec`
- La variable d'environnement `OMP_NUM_THREADS` permet de spécifier le nombre de threads OpenMP :
Option 1 : `% export OMP_NUM_THREADS=4 ; ./monprog`
Option 2 : `% OMP_NUM_THREADS=4 ./monprog`

I Problème : dynamique moléculaire

Le code fourni dans le répertoire `MolDyn/` est une maquette d'une simulation de dynamique moléculaire modélisant les interactions entre les molécules constituant un gaz.

Un `Makefile` permet de le compiler :

```
% make NPART=MINI => 1372 particules/molécules
% make NPART=MEDIUM => 4000 particules/molécules
% make NPART=MAXI => 13500 particules/molécules
```

L'exécution se fait avec `./md`.

Q.1: Étudier le code. Décrire son déroulement. Exécuter plusieurs fois le programme dans diverses configurations. Le résultat change-t-il entre deux exécutions ? Selon la configuration ? (Ne pas oublier de `make clean` avant de recompiler dans une configuration différente.)

La première étape dans la parallélisation d'un code est l'identification des régions les plus coûteuses. Nous allons utiliser `perf` afin de profiler la maquette (mode `NPART=MAXI`) :

```
% perf record -e cpu-clock ./md (génère un fichier perf.data)
% perf report (interprète le fichier perf.data présent dans le répertoire courant)
```

Q.2: Selon `perf report`, quelle est la fonction la plus coûteuse en terme de temps de calcul ? Paralléliser cette fonction. On portera une attention particulière au statut des différentes variables et aux potentiels problèmes d'accès concurrents.

Q.3: Faire des tests (en mode `NPART=MINI`) avec différentes politiques d'ordonnancement (utiliser `schedule(runtime)` et `OMP_SCHEDULE`) et avec différents nombres de threads. Rappeler le fonctionnement des différentes politiques et mettre en parallèle avec les résultats obtenus.

A partir de cette première version parallèle, nous allons maintenant également paralléliser la boucle extérieure (dans le fichier `main.c`) :

Q.4: Version 1 : les fonctions autres que la fonction `texttt` dans le corps de la boucle ne sont exécutées que par un seul thread.

Q.5: Version 2 : toutes les fonctions appelées dans le corps de la boucle sont parallélisées.

Les instructions `atomic` consistent un goulot d'étranglement sur la plupart des systèmes.

Q.6: Pour éviter l'utilisation des `atomic`, utiliser un tableau temporaire pour accumuler les forces avec une dimension supplémentaire indexée par le numéro du thread. Ne pas oublier l'accumulation finale.

Q.7: Faire une étude d'extensibilité en fonction du nombre de threads (en mode `NPART=MAXI`) .

TD à rendre sur ce lien (préciser `Prénom NOM`) avant 23h59 le 19/02/2026.

¹ <https://e.pcloud.com/#/puplink?code=4ML7ZeU4Ju9ANbDSotQmCT9YwxBjGjkkV>